

AWS Configurations

Log into AWS CloudShell directly via the official AWS Console:

- **AWS Console Login URL:** <https://console.aws.amazon.com/>
- **AWS CloudShell URL:** <https://console.aws.amazon.com/cloudshell/home>

Login with AWS root account and get Account ID: 877710816540 and sign out

Login with IAM user account with account ID: 877710816540

Select Region: us-east-1 N. Virginia

Check pip version and python version in PC

Check if pip (Python package installer) is working:

pip --version (in my PC pip 26.2.1)

python --version (in my PC python 3.14.7)

Install AWS CLI (Windows)

download the official 64-bit Windows MSI installer: and install AWS CLI

- **Direct Link:** [AWS CLI v2 Installer \(64-bit\)](#)

Configure AWS CLI

aws configure

Verify the Installation

Check AWS version

aws --version (Output: aws-cli/2.36.40 Python/3.14....)

Configure AWS CLI with Account Keys to Link AWS CLI to AWS Account

AWS Access Key ID [None]: Paste IAM User Access Key

AWS Secret Access Key [None]: Paste IAM User Secret Key

Default region name [None]:us-east-1

Default output format [None]:jsom

Aws configure list

aws sts get-caller-identity

Once configured, my PC is fully prepared to execute scripts and interact directly with AWS Bedrock.

Create Project folder and Project Files

Run these commands to create project folder and initialize a virtual environment:

```
cd C:\Users\Kumara
mkdir mail-clerk-agent
cd mail-clerk-agent
python -m venv venv
call venv\Scripts\activate
```

Run these commands to create empty files inside mail-clerk-agent:

```
type nul > config.py
type nul > email_service.py
type nul > ai_classifier.py
type nul > audit_logger.py
type nul > agent.py
type nul > lambda_handler.py
type nul > server.py
type nul > institutes.json
type nul > requirements.txt
type nul > .env
type nul > deploy.sh
type nul > deploy.bat
type nul > app.py
```

Install Dependencies

Install the required packages inside virtual environment:

```
pip install boto3 pydantic fastapi uvicorn python-dotenv
```

Save the exact package versions into requirements.txt:

```
pip freeze > requirements.txt
```

Insert the Code into Files

In the mail-clerk-agent folder, create following files and test with local host by running server

mail-clerk-agent/

- agent.py # holds the Core orchestrator logic
- lambda_handler.py # Entry point for AWS Lambda execution
- server.py # Optional API server for manual triggers/queues
- institutes.json # Mapping configuration for classification
- config.py # Loads AWS secrets and environment variables
- email_service.py # Handles IMAP (fetching) and SMTP actions (sending/drafting)
- ai_classifier.py # Interfaces with Amazon Bedrock (Nova Pro)
- audit_logger.py # Logs execution audit trails to CloudWatch
- deploy.sh # Deployment automation script
- deploy.dat #
- requirements.txt # Lists project dependencies
- Instruction.md # System instructions

Open project folder (C:\Users\Kumara\mail-clerk-agent) in text editor (Notepad) and paste the complete code into each corresponding file provided in My AWS Repo: https://github.com/kalugampitige/My_AWS_Repo

download all third-party Python libraries listed in your requirements.txt file and save them directly into a specific local folder named ./package.

```
pip install --target ./package -r requirements.txt
```

A shell script automating dependencies packaging, zipping, and AWS Lambda updates via the AWS CLI.

```
powershell Compress-Archive -Path * -DestinationPath ..\deployment_package.zip  
-Force
```

Executes the code inside deploy.bat

```
call venv\Scripts\activate
```

```
deploy.bat
```

command gives the script file (deploy.sh) executable permissions in AWS CloudShell

```
chmod +x deploy.sh
```

triggers the build and packages/uploads code to AWS

```
./deploy.sh
```

Run and Test Agent Locally

Run the local API server using uvicorn to make sure logic, and prompt pipelines work:

```
cd mail-clerk-agent
```

```
venv\Scripts\activate
```

```
uvicorn server:app --reload --port 8000
```

To run app.py install streamlit

```
pip install streamlit
```

Trigger Test Run via API Command: Open a second terminal window and run:

```
cd mail-clerk-agent  
curl -X POST http://localhost:8000/trigger-run  
venv\Scripts\activate  
streamlit run app.py
```

Direct Testing from Streamlit UI

- Open your Streamlit application at <http://localhost:8501>.
- Enter the prompt simulating the incoming message:

From: janethra127@gmail.com

Subject: Payment

Hi, please process invoice for billing.

- Press **Enter**.
- Streamlit triggers the updated `lambda_handler.py`.

Use Amazon SES(Simple Email Service) in AWS

To connect the agent to a real email inbox (to read incoming emails automatically) and send real destination emails (forwarding or replying), the standard AWS approach uses **AWS SES (Simple Email Service)** combined with **Amazon S3** and **SNS/Lambda**. Lambda uses the execution role (IAM Role) to send emails via SES.

[Incoming Email] —> AWS SES —> Saves Email to S3 —> Triggers Lambda —> Sends via SES Outbound

Send emails to verified addresses

- Open the **AWS Console** and search for **Simple Email Service (SES)**.
- Go to **Identities** → Click **Create identity**.
- Choose **Email address** (e.g., kalugampitige@gmail.com or personal Gmail) to start testing.
- Check your email inbox and click the verification link sent by AWS

Inside lambda_handler.py code, replace mock forwarding logic with the AWS SES send_email SDK action. I modified my original code by updating with this code.

```
import boto3
from config import load_institutes

# Initialize SES client safely
try:
    ses_client = boto3.client('ses', region_name='us-east-1')
except Exception as e:
    ses_client = None

VERIFIED_SENDER = "kalugampitige@gmail.com"
FORWARD_TARGET = "kalugampitige2@gmail.com"

# Built from institutes.json so it's always in sync - add/edit
institutes
# there, not here. Falls back to FORWARD_TARGET for anything unmapped.
DEPARTMENT_EMAILS = {
    inst["name"]: inst["contact_email"] for inst in load_institutes()
}
```

- **Routed Destination:** kalugampitige2@gmail.com is set as the recipient for Finance/IT requests in DEPARTMENT_EMAILS.

- **Verified Sender:** kalugampitige@gmail.com is retained as the Source address.
- **Logging:** Added log output inside send_email() to track delivery attempts in AWS CloudWatch logs.
- **Original Incoming Sender:** in addition to that I modified code by including janethra127@gmail.com

```
email_content = (
    f"--- AUTOMATED AI MAIL CLERK FORWARD ---\n"
    f"Original Sender: janethra127@gmail.com\n"
```

Here is the updated lambda_handler.py configured for exact flow:

- **Sender / Receiving Address:** kalugampitige@gmail.com
- **Forwarding Target:** kalugampitige2@gmail.com
- **Original Incoming Sender:** janethra127@gmail.com

Navigate to **AWS Lambda Console** → Functions → select mail-clerk-agent

→ Code tab → create following files in MAIL-CLERK-AGENT Folder

→ Paste code (Update from a .zip file: all files upload as .zip file) → Depoly

- agent.py # holds the Core orchestrator logic
- lambda_handler.py # Entry point for AWS Lambda execution
- institutes.json # Mapping configuration for classification
- config.py # Loads AWS secrets and environment variables
- email_service.py # Handles IMAP (fetching) and SMTP actions
- ai_classifier.py # Interfaces with Amazon Bedrock (Nova Pro)
- audit_logger.py # Logs execution audit trails to CloudWatch
- Instruction.md # System instructions

paste the complete code into each corresponding file provided in My AWS Repo:
https://github.com/kalugampitige/My_AWS_Repo

After Deploy(Ctrl+Shift+U) the code follow the setup below

- In Lambda function page, click the **Configuration** tab → select **Permissions** on the left menu.
- Click on the link under **Role name** (It takes to the IAM console).
- Click **Add permissions** → **Attach policies**.
- Search for AmazonSESFullAccess, check the box, and click **Add permissions**.

Got to lambda console(mail-clerk-agent) **Test tab** and click on **Test**

Open kalugampitige2@gmail.com and check the **Spam / Junk** folder and check the mail.

Send the real e.mail and Check agent workA

Agent check e.mail whether this keywords is in content or subject of e.mail. According to that e.mail Forward into relevant e.mail

```
{  
  
  "institute_id": "INST_FINANCE",  
  
  "name": "Finance & Billing Department",  
  
  "contact_email": "kalugampitige2@gmail.com",  
  
  "keywords": ["invoice", "payment", "receipt", "billing", "reimbursement"]  
  
},
```

```
{  
  "institute_id": "INST_HR",  
  "name": "Human Resources",  
  "contact_email": "janethra127@gmail.com",  
  "keywords": ["payroll", "vacation", "onboarding", "benefits", "policy"]  
},  
  
{  
  "institute_id": "INST_IT_SUPPORT",  
  "name": "IT Helpdesk",  
  "contact_email": "kalugampitige2@gmail.com",  
  "keywords": ["password", "hardware", "software", "access", "bug"]  
}
```

Samples of E.mail

From: janethra127@gmail.com

Subject: Request for Instructions

Check Email is received Correctly for payment Details. Thanks

From: janethra127@gmail.com

Subject: Request for Instructions

Dear Sir,

I would like to request a policy regarding the Settlement. Thanks

From: janethra127@gmail.com

Subject: Payment Query

Hi, please process invoice #4021 for billing.